

Programming in C

Unit	Unit Title	Total Marks
1	Basics of C Programming	12
2	Control Structures	16
3	Arrays and Structure	16
4	Functions	14
5	Pointers	12
Grand - Total		70

Assessment Scheme

Theory			Total Marks
FA-TH	SA-TH		
30	70		100
Practical			
FA-PR	SA-PR	SLA	
Max Min	Max Min	Max Min	
50 20	50# 20	25 10	225
			225

By - Anuraag, Khushi, Preeti Ma'am

Unit 1 Basics of C programming

1-1 Fundamentals of Algorithms

* **Algorithms:** An algorithm is a procedure used for solving a problem or performing a computation.

Algorithms act as an exact list of instructions that conduct specified actions step by step in slow or fast.

Algorithms is a step by step method of solving a problem.

Properties of an Algorithm

1. **Input:** An algorithm accepts zero or more inputs.

2. **Output:** An algorithm should produce at least one output.

3. **Finiteness:** An algorithm terminates after a finite number of steps.

4. **Definiteness:** Each step in algorithm is unambiguous.

This means that the action specified by the step cannot be interpreted in multiple ways and can be performed without any confusion.

5. **Effectiveness:** Each step must be basic enough to be executed.

* **Pseudocode:** It is an artificial and informal language that helps programmers develop algorithms. Pseudocode is very similar to everyday English.

- **Get:** used to get information from the user

- **Display:** used to display information for the user

- **Compute:** perform an arithmetic operation (+, -, *, /)

- **Store():** store a piece of information for later use.

* Algorithms to find sum & product of 2 nos as follows.

- Step 1: Read 2 numbers a, b from keyboard
- Step 2: Let $\text{sum} = a + b$
- Step 3: Let $\text{product} = a * b$
- Step 4: print sum, product
- Step 5: Stop

* Algorithms to find area of circle.

- Step 1: Read the value of radius R
- Step 2: Calculate area using formula: $\text{area} = 3.14 * R * R$
- Step 3: Print the value of area.
- Step 4: Stop.

* Algorithms to determine the given number is odd or even

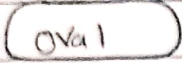
- Step 1: Read a number from keyboard
- Step 2: if $n \% 2 == 0$ then number is even.
- Step 3: else number is odd
- Step 4: print the output.
- Step 5: Stop.

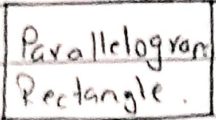
* Flow-chart : Flow-chart is a pictorial representation of steps of algorithm used to solve the problem.

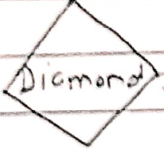
The flowchart shows the steps as boxes of various kinds, and their order by connecting the boxes with arrows.

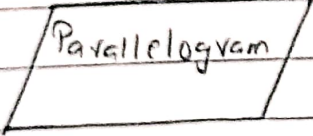
A flow-chart is a type of diagram that represents a workflow or process.

Various flowchart symbols are:

1)  Start & end Symbol

2)  Processing Symbol

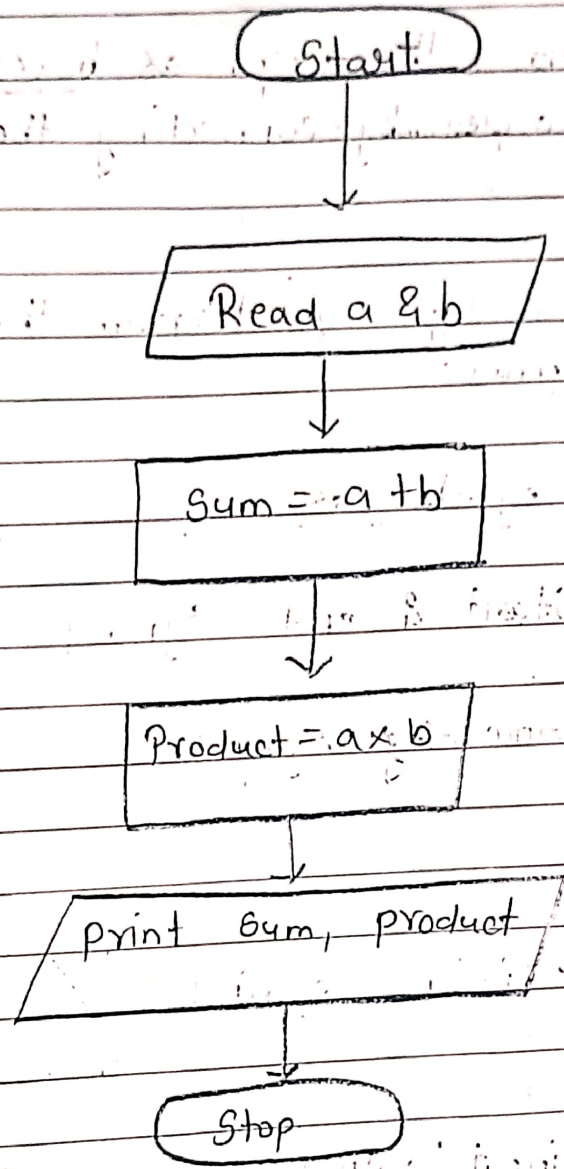
3)  Decision Symbol

4)  input/output Symbol

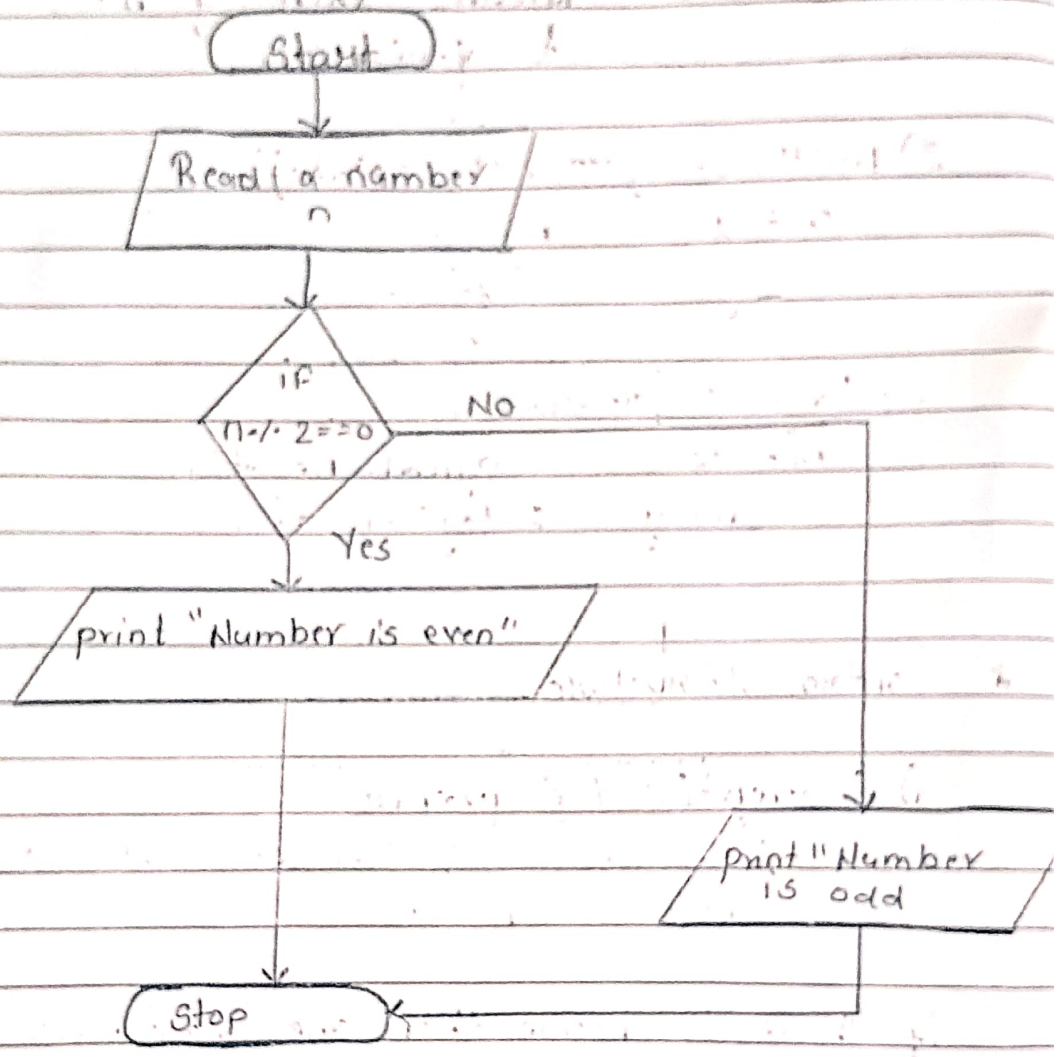
5)  Connector Symbol

6)  Flow Symbol
Arrows.

Q. Draw a flow chart to find sum & product of 2 numbers



* Draw flowchart for checking whether given number is even or odd.



* Notation used:

1) Assignment Statement:

$a = b + 5;$ means value of $b + 5$ is assigned to variable a .

2) Arithmetic Operator Notation:

$x + y$: Addition

$x - y$: Subtraction

3) Logical operator:

$\text{not } x$: logical negation

$x \text{ and } y$: logical and

* Control Structure

1) Sequential / Sequence

Statements are executed one after another in the order they are written.

If there is more than one statement in the line, the statements are separated by a semicolon (;)

E.g: $\text{int } a = 5;$
 $\text{int } b = 10;$

2) Selection:

Used when the program must choose between two or more alternative.

E.g: if (age $\geq$ 18)

{

printf("Eligible");

}

3) Iteration / Repeation

It is used when a statement needs to be repeated multiple times.

E.g: For (int i = 1; i $\leq$ 5; i++)

{

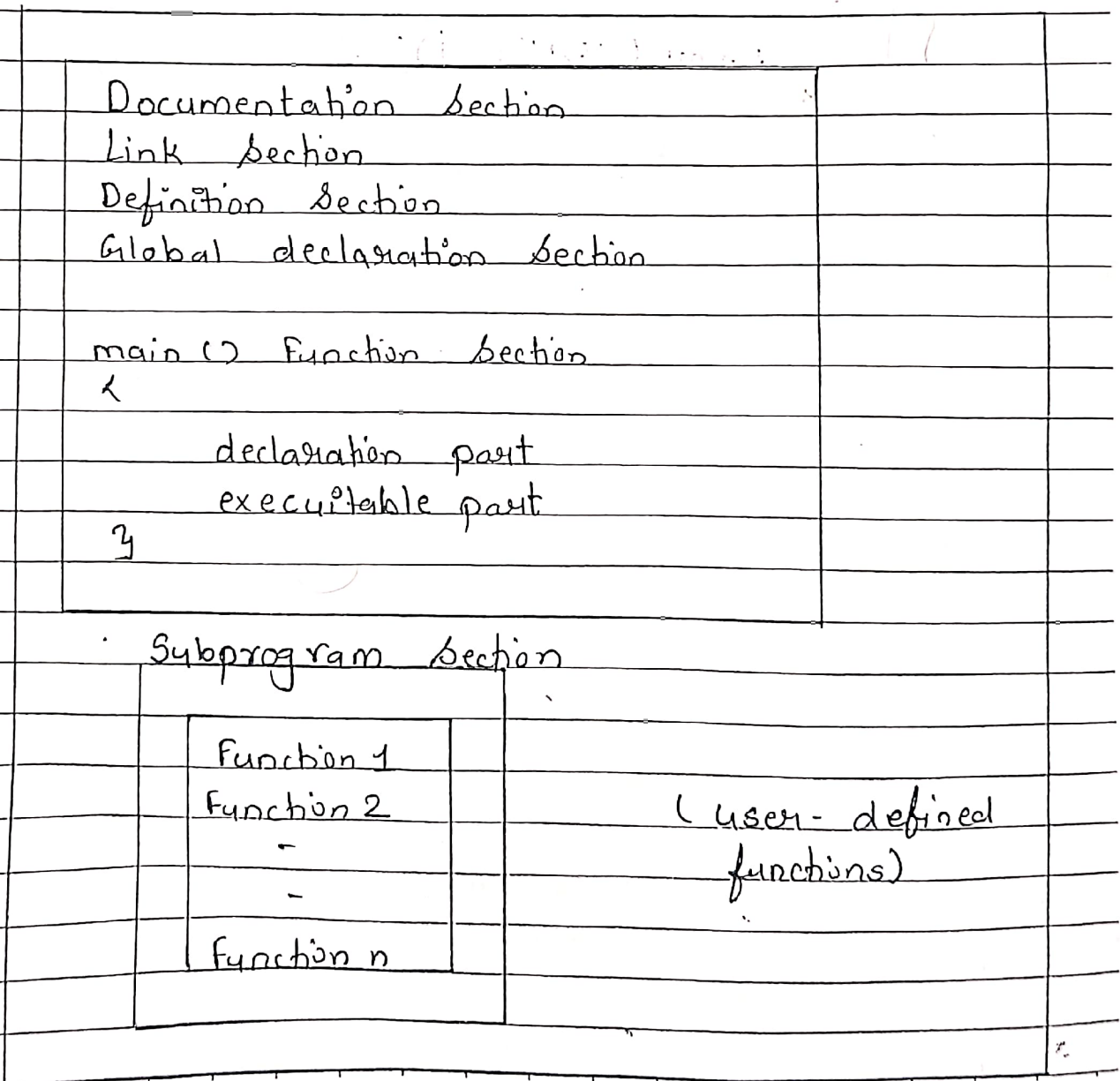
printf("%d", i);

}

* 1-2 Introduction to C

C is a programming language developed at AT & T's Bell Laboratories of USA in 1972. It was designed and written by a man named Dennis Ritchie. In the late seventies C began to replace the more familiar language of that time like PL/I, ALGOL, etc.

General structure of C



1) Document Section: It consists of a lot of C++ comment lines giving the name of the program and other details.

2) Link Section: It provides instructions to the compiler to link function from the system library.

3) Definition Section: It defines all symbolic constants.

4) Global Declaration Section: Declaring variables outside all function in a program. It can be accessed and used by any function in the program.

5) main() function section: This section contains two parts: declaration part & executable part. The declaration part declares all the variables used, executable part the program execution begins at the opening brace and ends with closing brace.

* Header File

There are various header files are:-

i) `stdio.h`

ii) `math.h`

iii) `dos.h`

iv) `conio.h`

These files should be included as

```
#include <stdio.h>
```

```
#include <conio.h>
```

* main() Function

The `main()` is a special function used by the C system to tell the computer where the program starts. Every program must have exactly one main function. If we use more than one main function, the compiler cannot tell which one marks the beginning of the program.

1.3 Fundamental Construction of C

C Character Set

An alphabet, digit or any special symbol used to represent information is called character.

Valid characters allowed in C are as follows:

1) Alphabet

A, B, C, ..., X, Y, Z

a, b, c, ..., x, y, z

2) Digits

0, 1, 2, 3, 4, 5, 6

3) Special Symbols

~ ! @ # % , < > ? [] ^ _

* White spaces are ignored by compiler until they are part of string constant.

* Tokens in C:

The smallest individual units in C are known as C tokens.

A token is a keyword, identifier, constant, string, operators etc.

C has six types of tokens

Tokens

- ① Keywords ② identifiers ③ Constants ④ Strings ⑤ Special ⑥ Operator

Example of each token as follows.

1) Keywords - if, else, main, while

2) identifiers - a, sum, amt, avg-of-no.

3) Constant - 10, 20.5, 'A', 'welcome'

4) Strings - "ABC", "year"

5) Special symbol - [], < >, \$

6) Operator - +, -, <, >, &&, &, <<, >>

* Keywords:

Keywords are the words whose meaning is already explained to the C compiler. They have fixed meaning and these meaning cannot be changed. The keyword cannot be used as variable name or function name.

Keywords are: (C language specifies 32 keywords)
auto, double, int, break, printf, scanf, for, if, do, else, long, while, void, float, case,

* Constants:

A Constant is an entity that does not change during execution of a program. Constants can be different types as integer constants, real constants, character constants, string constants etc.

Integer Constant: 612, 3.14, -71

Character Constant: 'y', '@', '2'

Escape Sequence: \n (newline), \b (Backspace)

String Constant: "Hello", "15 August"

* Identifiers:

It refers to the name of variable, function, array etc. They are user defined names.

C is case sensitive, i.e., it identifies difference between lowercase (small) and uppercase (capital) letters. So, all the identifiers will be treated as different identifiers.

val, Val, VAL, vAl, VaL

* Variables:

Variables is an entity whose value can be changed during execution of a program. Variable names are the names given to location in memory (primary memory which is most commonly RAM).

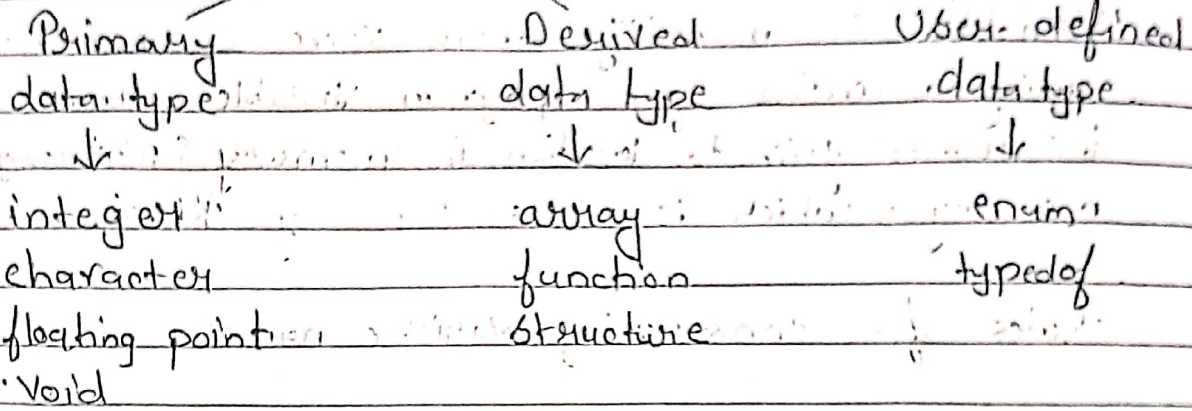
Rules for constructing variable name:

- ① First character must be letter or underscore
a, b10, sum, average etc.
- ② Variable must be consist of A-Z, a-z & 0-9 digits
- ③ Only special symbol underscore (_) is allowed in variable.
- ④ Maximum length of 31 characters
- ⑤ Variable should not be any keywords like else, if, while etc.
- ⑥ No blank space or comma are allowed in variable name.

* Data types in C:

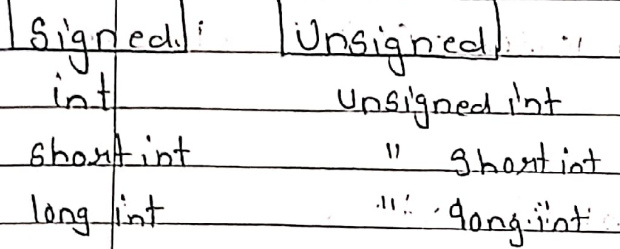
Data types specifies what type of data can be stored in a variables.

Data Types



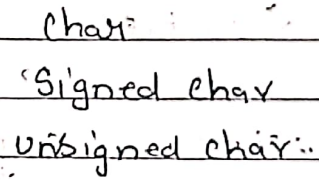
Primary Data type

1 Integer type

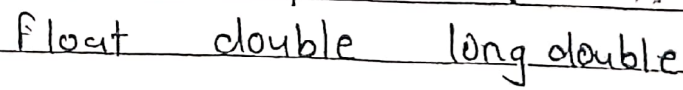


3

Character



2 Floating point



4 Void

-32768 to 32767
0 to 255

The main data types and its size and range as follows.

Data types	Size in bytes	Range
int	2	-2^{15} to $+2^{15} - 1$
long int	4	-2^{31} to $+2^{31} - 1$
float	4	$3.4 \text{E} - 38$ to $3.4 \text{E} + 38$
double	8	$1.7 \text{E} - 308$ to $1.7 \text{E} + 308$
char	1	-2^7 to $2^7 - 1$

* Declaration of Variable

A variable is used to store value of any data type.

Syntax: data-type variable-name;

E.g : int a, b, c;

char choice;

float area;

When a variable is declared, memory space is allocated and the variable points to the allocated memory.

The variable can be declared in two parts

- i) Outside all functions (Global declaration) See
Variable declared here are called global variable. These variables are accessible or visible to all the functions. Scope of this variable is global.

ii) Immediately after '`{`' in a block if these variable are local variable, they are accessible or visible within the block where they are declared. Scope of such variable is local.

Variable Initialization

When we assign any initial value to variable during the declaration is called initialization of variables. When variable is declared but contain undefined value then it is called garbage value. The variable is initialized with the assignment.

Syntax: `datatype variable name = Constant;`
 E.g: `int a = 20;`

Expressions :

An expression is a combination of variables, constant, operators and function call. It can be arithmetic, logical and relational.

Example :- `int z = x + y` // arithmetic expression
`a > b` // relational
`a == b` // logical
`Func(a, b)` // function call

Data-type Conversion

In some situations, programmer needs to convert data type of a variable, a value or an expression.

temporarily. Process of doing so is called typecasting.

Syntax:-

```
int a, b;
```

```
float avg;
```

```
printf("Enter two no");
```

```
scanf("%d %d", &a, &b);
```

```
avg = (float) (a+b) / 2;
```

/* Values (a+b) & 2 integer both numbers

& denominator are integer, division operation

gives integer result. To get the floating

point result, either of them needs to

be made float */

1-4 Basic Input and Output Function

Input: In any programming language input means to feed some data into program. This can be given in the form of file or from command line.

Output: In any programming language output means display some data on screen, printer or in any file.

These functions allow the transfer of information between the computer and the input/output device.

Input Function & Output Function

C language has standard libraries that allow input and output in a program.

The `stdio.h` or standard input output library in C that has methods for input and output.

`scanf()`: in C reads the value from the console as per the specified.

`printf()`: in C, prints the value passed as the parameter to it, on the console screen.

The Syntax for input and output :-

Integer :-

Input :- `scanf("%d", &intVariable);`

Output :- `printf("%d", intVariable);`

Float :-

Input :- `scanf("%f", &FloatVariable);`

Output :- `printf("%f", FloatVariable);`

Character :-

Input :- `scanf("%c", &charVariable);`

Output :- `printf("%c", charVariable);`

// C program to show input & output.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    // Declare the variables
```

```
    int num;
```

```
    char ch;
```

```

Float F; // Input the float
// -- Integer --
// Input the integer
printf("Enter the integer:");
scanf("%d", &num);

// Output the integer.
printf("\n Entered integer is: %d", num);

```

```

// -- Float --
// Input the Float

```

```

printf("\n\n Enter the float:");
scanf("%f", &F);

// Output the float.
printf("\n Entered Float is %f", F);

return 0;

```

Output:

Enter the integer: 10

Entered integer is: 10.

Enter the float: 2.5

Entered Float is: 2.500000.

scanf function, the arguments is a

scanf("%d", &num)

Here,

%d → Format specifier for integer

&num → address of variable where input will be stored.

Format Specifier

Conversion character	Meaning
%c	to read a single character
%d	to read a decimal integer
%f	to read a floating point value
%g	to read a floating point value
%s	to read a string value
%o	to read a decimal octal value
%h	to read a short integer
%u	to read unsigned decimal integer
%e	to read a floating point value with exponent value

scanf() function required different format specifiers (also known as format codes) to read the data.

Similarly, printf() function required different format specifier to print data on the screen.

1.5 Assignments and expressions

* Assignments

A variable can be assigned some value after its creation.

For example: `int a, float b`

`a = 10`

`b = 3.4`

Here,

variable `a` & `b` are created first and values to them are assigned after some time.

* Expressions

An expression is a combination of variable constants operators and function call. It can be arithmetic logical and relational.

E.g:- `int z = x + y` // arithmetic expression

`a > b` // relational

`a == b` // logical

`func(a, b)` // function call.

expressions {

`z = x + y`

`z = 5 + 5`

Variables ←



Operators

* Operator:

This is a symbol use to perform some operation on variable (operands) or with the constant. Some operator required 2 operands to perform operation or some required single operation.

The operator are divided into the following types

- 1) Arithmetic Operator
- 2) Assignment Operator
- 3) Increment and Decrement
- 4) Relational Operator
- 5) Conditional Operator
- 6) Bitwise operator

1) Arithmetic Operator

Arithmetic

1. Binary operators

2. Unary Operators

This operator used for numeric calculation.

1. Binary Operators

Binary operators are the operators which always work with two value (i.e. two variable).

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder

Example:-

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
int a, b, c;
```

```
printf("Enter the value of a, b");
```

```
scanf("%d %d", &a, &b);
```

```
c = a + b;
```

```
printf("Addition of a, b is = %d\n", c);
```

```
c = a - b;
```

```
printf("Subtraction of a, b is = %d\n", c);
```

```
c = a * b;
```

```
printf("multiplication of a, b is = %d\n", c);
```

```
c = a / b;
```

```
printf("Division of a, b is = %d\n", c);
```

```
}
```

2. Unary Operators:

Operator	Purpose
-	Negative value
+	Positive value
++	Preincrement, Postincrement
--	Predecrement, Postdecrement
sizeof()	Find size of variable in terms of memory.

Unary operators are usually written before variable, though some unary operators are written after their operands.

Unary operators that perform operation on a single variable to produce a new value.

2) Assignment Operator

A value can be stored in a variable with the use of assignment operator. The assignment operator ($=$) is used in assignment statement and assignment expression. Operands on the left hand side should be variable and the operand on the right hand side should be variable constant or any expression. When variable on the left hand side is occur on the right hand side then we can avoid by writing the compound.

For example `int x = y;`

`int sum = x + y + z;`

3) Increment & Decrement Operator

Increment Operator

- $++$ is called increment operator.
- It is an unary operator and written as follows
 $++i$ OR $i++$
- The $++$ adds 1 to the operand i.e. it is same as $i = i + 1$.
- Increment operator has two form
 - 1) Pre-increment ($++i$)

2) Post-increment (b++)

Example:

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
int a, b, a1, a2;
```

```
a = 10;
```

```
b = 10;
```

```
a1 = ++a; // Pre-increment
```

```
b1 = b++; // Post-increment
```

```
printf("\n a1 = %d", a1);
```

```
printf("\n b1 = %d", b1);
```

```
getch();
```

```
}
```

It will give the output as follows.

```
a1 = 11
```

```
b1 = 10
```

When we write statement

```
b1 = b++;
```

In this case, value of b (i.e. 10) is first assign to b1 and then value of b is incremented by 1 (i.e. it becomes 11).

Decrement Operator

- 1) `--` is called decrement operator.
- 2) It is an unary operator and substitutes as follows

`--i` or `i--`

- 3) The operator `--` subtracts 1 from the operands i.e. `--i` is as `i = i - 1`.

- 4) Decrement operator is in two form

- i) pre-decrement (`--i`)
- ii) post-decrement (`i--`)

- 5) Example :-

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
int a, b, a1, b1;
```

```
a = 10;
```

```
b = 10;
```

```
a1 = --a;
```

```
b1 = b--;
```

```
printf("In a1 = %d", a1);
```

```
printf("In b1 = %d", b1);
```

It will give the o/p as follows

```
a1 = 9
```

```
b1 = 10
```

In this case, value of b (i.e. 10) is assign to b1 first and then value of b is decremented by 1. i.e. it became 9.

4) Relational Operator

It is use to compared value of two expressions depending on their relations. Expression that contain relational operator is called 'relational expression'.

Operator	Description	Syntax
>	Greater than: True if left operand is greater than the right.	$x > y$
<	Less than: True if left operand is less than the right.	$x < y$
==	Equal to: True if both operands are equal.	$x == y$
!=	Not equal to - true if operands are not equal.	$x != y$
>=	Greater than or equal to True if operands is greater than or equal to the right.	$x >= y$
<=	Less than or equal to: True if left operand is less than or equal to the right.	$x <= y$

5 Conditional Operator.

The operator pair "?:" is called conditional operator.

- It is also called ternary operator.
- The general form is

exp1	?:	exp2	:	exp3
------	----	------	---	------

where, $exp1$, $exp2$ & $exp3$ are expressions

- In this, $exp1$ is evaluated first. If value of $exp1$ is true (i.e. non zero) then pc goes to execute $exp2$ otherwise pc goes to execute $exp3$.

Example:-

```
void main()
```

```
{
```

```
    int a, b, max;
```

```
    a = 30;
```

```
    b = 20;
```

```
    c = 10;
```

```
    max = (a >= b) ? a : b;
```

```
    printf("Maximum no = %d", max);
```

```
}
```

6 Bitwise Operator

C programming language support following bitwise operators

Operator	Meaning
&	bitwise AND
	OR
^	exclusive OR
<<	Shift left
>>	Shift right

& operator: This operator is used to perform bitwise and rules are as follows:

x	y	x & y
0	0	0
0	1	0
1	0	0
1	1	1

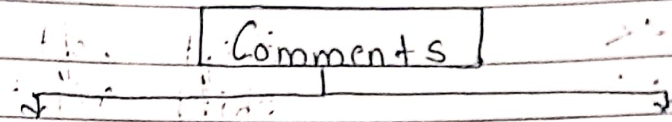
| operator: This operators is used to perform bitwise binary OR and it rules are as follows:

x	y	x y
0	0	0
0	1	1
1	0	1
1	1	1

* Comments in C

A Comment is a programmer-readable explanation or annotation in the source code of a computer program.

Comments are statements that are not executed by the compiler and interpreter.



1. Single line Comments

2. Multi-line Comments

* Single line comments

1. Represented as // double forward slash
 2. It applies comment to a single line only
- // Single line comment

* Multi-line comments

1. Represented as /* any-text */ Start with forward slash and asterisk (/*) and end with asterisk and forward slash (*/).
2. It is used to denote multi-line comment. It can apply comment to more than a single line.

/* Comment starts

continues

continues

:

:

Comment ends */